A Survey on Optimized Implementation of Deep Learning Models on the NVIDIA Jetson Platform

Sparsh Mittal
IIT Hyderabad.
E-mail:sparsh@iith.ac.in.

Abstract

Design of hardware accelerators for neural network (NN) applications involves walking a tight rope amidst the constraints of low-power, high accuracy and throughput. NVIDIA's Jetson is a promising platform for embedded machine learning which seeks to achieve a balance between the above objectives. In this paper, we provide a survey of works that evaluate and optimize neural network applications on Jetson platform. We review both hardware and algorithmic optimizations performed for running NN algorithms on Jetson and show the real-life applications where these algorithms have been applied. We also review the works that compare Jetson with similar platforms. While the survey focuses on Jetson as an exemplar embedded system, many of the ideas and optimizations will apply just as well to existing and future embedded systems. It is widely believed that the ability to run AI algorithms on low-cost, low-power platforms will be crucial for achieving the "AI for all" vision. This survey seeks to provide a glimpse of the recent progress towards that goal.

Index Terms

Review, embedded system, NVIDIA Jetson, neural network, deep learning, autonomous driving, drone, low-power computing

1 Introduction

In recent years, machine learning and especially neural network (NN) algorithms have attracted significant amount of attention from the researchers. NN algorithms have achieved high accuracy in many fields such as image processing, natural language processing, data analytics, etc. These successes have motivated the researchers to apply NN algorithms to solve increasingly complex tasks touching every aspect of human life.

However, to become the part of life of every citizen, the NN-based smart solutions must pass the test of energy-efficiency, small form-factor and affordability. The urgency of achieving these objectives is also evident from the introduction of recent competitions such as "low-power image recognition challenge" (LPIRC) [1, 2] which emphasize a balance between accuracy, throughput and *power budget*. These objectives are not merely attractive but even imperative in several domains such as internet-of-things (IoT), robotics, autonomous driving, drone-based surveillance, etc. Further, deep-learning algorithms were conventionally executed in the centralized cloud environment. This cloud-computing approach, however, incurs large latency, energy and financial overheads and also raises privacy concerns. To mitigate these challenges, edge-computing approach has been proposed where computation on the data is performed locally, i.e., where the data is produced, using edge devices. Edge devices are especially promising for accelerating deep-learning algorithms due to their low power-budget and high efficiency. For fulfilling these computational demands, both major vendors and startups have designed and launched low-power hardware accelerators for machine learning.

- *This work was supported in part by Semiconductor Research Corporation (SRC).*

Among these, NVIDIA's Jetson is very promising and one of the most widely used accelerators for the inference phase of machine learning. Jetson features CPU-GPU heterogeneous architecture [3, 4] where CPU can boot the OS and the CUDA-capable GPU can be quickly programmed to accelerate complex machine-learning tasks. Further, it has small form factor, low weight and power consumption (refer Section 2.1) which makes it a perfect fit for weight/power-constrained scenarios. However, leveraging the full potential of Jetson and achieving real-time performance requires performing optimizations to both Jetson hardware and NN algorithms. Many recent works have sought to address these challenges.

Contributions: In this paper, we present a survey of works that evaluate and optimize neural network applications on Jetson platform. Figure 1 provides an overview of the paper. Section 2 shows architectural parameters of Jetson and related embedded systems. It also presents motivation for using embedded systems for machine learning and classifies the works based on key parameters. Section 3 shows the architectural optimizations performed on Jetson to bring out its best and match its resources to the characteristics of NN algorithms. Section 3 also discusses the works that compare Jetson with similar systems, such as Raspberry Pi, FPGA, Intel NUC, etc., to highlight their advantages and limitations.

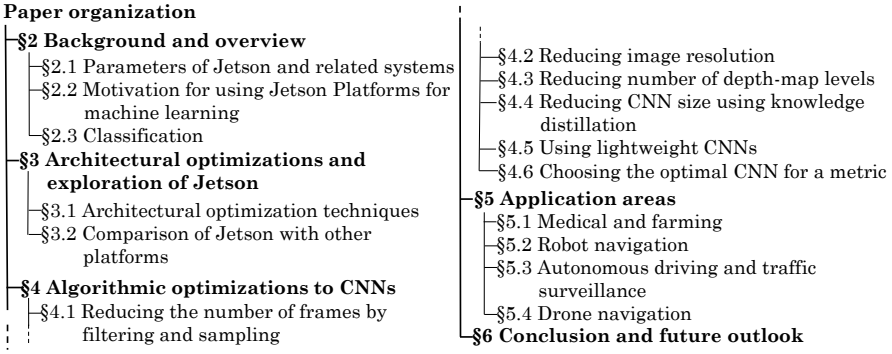


Fig. 1. Organization of the paper

Section 4 reviews the optimizations applied to NN algorithms to exercise a tradeoff between accuracy and performance on embedded systems. Section 5 reviews the techniques in terms of the real-life applications in which they have been deployed. In these sections, we discuss a technique in a single category only, even though most of the works span across the categories. Section 6 presents concluding remarks and avenues for future research.

While the survey focuses on Jetson as a representative embedded system, many of the insights provided by this survey will also be useful for the design of future low power-budget processors and development kits. The survey seeks to show how the choice of machine learning algorithm is made not only based on its accuracy but also based on the resource-constraints of the hardware platform. In fact, hardware platform can force fundamental changes in the algorithm design. The survey bears testimony that hardware-software co-design is more relevant today than ever before. Overall, this survey aims to form a bridge between researchers in artificial-intelligence community and computer-architecture community.

Scope: We do not include works that compare Jetson with research prototypes and chip-level demonstrations, since their results cannot be reproduced and the research designs do not take into consideration all the challenges faced in the design of commercial products. Also, we do not focus on works where the computation is performed in the cloud and the embedded system is used merely or primarily for computation offloading.

We use the following acronyms frequently in this paper: convolution/deep/recurrent neural network (CNN/DNN/RNN), field programmable gate array (FPGA), frames per second (FPS), fully-connected (FC), light density and ranging (LIDAR), long short term memory (LSTM), network/system on chip (NoC/SoC), proportion-derivative (PD), proportion-integral-derivative (PID), rectified linear unit (ReLU), reduced instruction set computer (RISC), single shot multibox detector (SSD) [5], support vector machine (SVM), you only look once (YOLO) [6].

2 BACKGROUND AND OVERVIEW

In this section, we discuss architectural parameters of Jetson and related systems (Section 2.1) and motivation for using Jetson for accelerating machine learning applications (Section 2.2). We then present a classification of research works on key metrics (Section 2.3).

2.1 Parameters of Jetson and related systems

Four models of Jetson have been released which are termed TK1, TX1, TX2 and Xavier. Of these, Xavier has been released very recently and has not been widely used in research. Hence, in this survey, we focus on the works that use the first three models only. Table 1 shows their parameters [7–10]. For sake of comparison, Table 1 also shows the parameters of related embedded systems, viz., Raspberry Pi and Intel UP. For configuration of other similar systems, we refer the reader to previous works [11–14].

TABLE 1
Selected parameters of Jetson, Raspberry Pi and Intel UP (the prices are as of Dec 2018. DP/SP = double/single-precision). For Jetson, peak performance is GPU's performance, whereas for Raspberry, it is CPU's performance. These systems also have other accelerators which are not shown.

	TK1	TX1	TX2	Raspberry Pi 3(B+)	Intel UP
Feature size	28nm	20nm	16nm	40nm	data not found
GPU	192-core Kepler	256-core Maxwell @ 998MHz	256-core Pascal @ 1300MHz	VideoCore IV	Intel HD 400 Graphics, upto 500 MHz
CPU	"4-Plus-1" 2.32GHz ARM quad-core Cortex-A15 CPU with Cortex-A15 battery-saving shadow-core	ARM Cortex-A57 (quad-core) @ 1.73GHz	ARM Cortex-A57 (quad-core) @ 2GHz + NVIDIA Denver2 (dual-core) @ 2GHz	Broadcom BCM2837B0 quad-core ARM Cortex-A53 @ 1.4GHz	Intel Atom x5-Z8350 quad-core CPU, 64-bits @ 1.92GHz
Memory	2GB DDR3L 933MHz EMC x16 using 64-bit data width	4GB 64-bit LPDDR4 @ 1600MHz, 25.6 GB/s	8GB 128-bit LPDDR4 @ 1866Mhz, 59.7 GB/s	1GB LPDDR2 @ 900MHz, 8.5 GB/s	4GB DDR3L @ 1600 MHz
Storage	16 GB eMMC	16 GB eMMC	32GB eMMC	MicroSDHC slot	16/32/64 GB eMMC
Peak performance	>300 SP Gflops [15]	512 SP Gflops [16]	665 SP Gflops [16]	6 DP Gflops [17, 18]	data not found
Power under load	10W	1W to 15W [19]	7.5W to 15W	1.5W to 6.7	6W
Weight	120 gram [20]	85 gram (with thermal transfer plate) [21]	85 gram (with thermal transfer plate) [21]	50 gram [22]	80 gram (with passive heat sink but without package) [23]
Price	Discontinued	$480	$570	$35	$100

2.2 Motivation for using Jetson Platforms for machine learning

Despite their limited capabilities, embedded computing systems in general and Jetson in particular are very useful as machine-learning accelerators, for following reasons.

Enabling local-processing: Due to their low-power consumption, embedded systems allow processing the data at the site where it is collected, e.g., an IoT device, a robot, an autonomous-car or a drone. Local-processing offers several advantages over offloading the computations to a remote server or data-center. Firstly, offloading incurs high latency, energy, financial and computation infrastructure overheads. It also requires a connection with high bandwidth and reliability, since the data-transmission may not complete in case of poor signals. Secondly, data-transmission leads to security and privacy issues. Finally, local-processing can reduce the volume of data transmitted to the cloud which allows it to perform high-level tasks.

Low weight and power consumption: In many application domains, having a low weight is a crucial requirement of deployment for any computing system. For example, the power required to hover a drone of mass M increases with $M^{1.5}$ [24]. Hence, a lightweight computing system can increase the battery life and flight time. It is well-known that for each watt of power dissipated in a computing system, at least 0.5 watt additional power need to be spent on cooling system [25]. Hence, a low-power computing system

can reduce thermal issues and cooling requirement, which is especially important in domains such as autonomous driving vehicles. Both the weight and power consumption of Jetson are small. Similarly, by virtue of its small form factor, it can easily fit even in a toy-car, a robot or a drone.

Unique features of Jetson: Even among embedded systems, Jetson has become especially popular due to its unique features. It provides high performance per watt due to its performance-efficient and low-power GPU cores. In general, GPUs provide higher throughput than FPGAs [26], which makes Jetson suitable for deep-learning applications. From Table 1, it is evident that Jetson provides higher peak performance than Raspberry Pi, Intel UP board and Intel Movidius stick [27]. Further, the GPU in Jetson is CUDA-programmable. Since DNN applications trained on high-end GPUs are also coded using CUDA, these applications can be (generally) directly ported to Jetson for inference. TX1 and TX2 also support cuDNN library which provides optimized implementation of important kernels. This facilitates implementing DNNs efficiently. By comparison, for other devices such as Raspberry Pi and FPGA, deep-learning kernels need to be developed using OpenCL, since a cuDNN-like library is not available for them [7].

2.3 Classification

Table 2 classifies the research works based on the Jetson model and the machine-learning algorithm used by them. Table 2 further classifies the works based on their study/optimization metric and the FPS achieved. Evidently, many research works do not achieve the frame rate required for satisfactory visual experience. Xavier, the latest model of Jetson, claims to provide 20 times higher performance and 10 times higher energy efficiency compared to TX2 [28]. Thus, it is expected that Xavier will allow achieving real-time frame-rate for more complex tasks. Further classification on several other metrics is presented in subsequent sections.

3 Architectural optimizations and exploration of Jetson

In this section, we review works that perform CPU and/or GPU architecture-level optimizations (Section 3.1) and compare Jetson with other computing systems (Section 3.2).

3.1 Architectural optimization techniques

Table 3 summarizes the hardware-level optimizations performed by different works. In the heterogeneous computing approach, the workload is divided between the processors with disparate architecture, viz. the CPU and GPU, to bring the best of two together [101]. Some works perform memory-level optimizations to GPU, such as using shared and texture memory, reducing accesses to global memory by using memory access coalescing and kernel fusion, etc. Some works scale the frequency of CPU and/or GPU to tradeoff performance with energy [102]. These works exploit slack in application deadline or latency-tolerance of end-user to reduce frequency. For example, during idle phases, the frequency can be lowered for saving energy, whereas to meet a task-deadline, the frequency can be increased. Many techniques exploit the error-tolerant nature of humans and neural network algorithms [103] to tradeoff accuracy for achieving efficiency. These techniques are also shown in Table 3. We now review a few of them.

In the "low-power image recognition challenge" (LPIRC), the goal is to maximize the ratio between accuracy and energy consumed in processing 20,000 images within 10 minutes. If the processing of 20,000 images is finished before 10 minutes, the energy consumed till that time is computed. If less than 20,000 images are processed in 10 minutes, the ratio above is scaled by the number of completed images.

Kang et al. [7] discuss their CNN architecture which was ranked first in the LPIRC 2017 competition. As for the hardware platform, they qualitatively compare five platforms, viz., TX2, TX1, Exynos 5422, Exynos 8895 and Snapdragon 835. From these, they select TX2 since it provides the highest throughput. As for the object detection algorithm, they use Tiny-YOLO since its product of accuracy and frame-rate is higher than that of YOLO and SSD.

To further improve efficiency, they propose several optimizations. Since not all predictions of Tiny-YOLO are accurate, a post-processing step is performed which selects predictions with more than a threshold value of confidence. This step is performed on CPU. First, they pipeline the execution of

TABLE 2
A classification of research works

Category	References
Jetson model used	
TK1	[11, 29–40]
TX1	[12, 41–56]
TX2	[7, 24, 48, 54, 57–84]
Algorithm used	
Neural network	RNN [85, 86], CNN (nearly all)
SVM	[54, 64, 65]
k-nearest neighbor	[54, 65]
k-means clustering	[77]
PID or PD controller	[61, 82]
Decision tree	[24, 65]
Evaluation/optimization objective	
Accuracy or speed	nearly all
Power/energy	[12, 29, 32, 38, 42, 44, 45, 47, 54, 55, 65, 72, 87–89]
CNNs used	
AlexNet	[12, 13, 30, 39, 47, 51, 56, 81, 90]
YOLO	[32, 41, 43, 49, 67, 79, 82, 91–93]
YOLO-tiny	[7, 80, 92–94]
ResNet	[55, 63, 65, 68, 73, 95]
MobileNet	[65, 66, 73, 77, 87, 96–98]
SSD	[81, 87, 98]
VGG	[39, 56, 62, 73, 99]
Others	SqueezeNet [13, 76], Fast RCNN [52], Faster RCNN [52, 75, 76], SegNet [36, 58], CNN used in NVIDIA's DAVE-2 car [70, 85], VGGM [99], ERFNet [84], ZFNet [99], InceptionNet [65, 73], FlowNet [100], XceptionNet [73], U-Net [57]
FPS achieved	
≤ 5	[31, 32, 62, 67, 97]
5 to 10	[69, 74, 93, 94, 98]
10 to 20	[24, 30, 60, 68, 83, 84, 91]
20 to 30	[33, 41, 43, 73, 75]
≥ 30	[34, 35, 42, 46, 53, 57, 64, 70, 78, 80]

TABLE 3
A classification based on hardware architecture-level optimizations

System and CPU-level	CPU-GPU heterogeneous computing [32, 37, 72], pipelining [7, 32, 42, 44, 92], CPU frequency scaling [7, 37], multithreading on CPU [7, 73]
GPU-level	GPU frequency scaling [7, 32, 37], use of TensorRT [72, 78], kernel fusion for reducing global memory transfers [38], executing kernels concurrently on different CUDA streams [38, 40], memory access coalescing [38], use of texture cache/memory [42, 91] and shared memory [38–40], CUDA managed memory [32]
Approximate computing approaches	Using partial face image instead of entire face image [30], using image of only one-eye instead of both eyes, based on their symmetry [30], working on videos with only small change between different frames [104], dropping unused row in convolution [104], quantization [7, 92], reducing bitwidth/precision [38, 52, 55, 72, 78]

pre-processing and post-processing steps on CPU with inference step on the GPU. Second, they apply Tucker-decomposition in convolution layers. This is an approximate computing approach which reduces the number of multiplications by using 1×1 convolution. However, they note that Tucker decomposition does not reduce execution time in all the layers. Hence, they apply it in only those convolution layers where its reduces the execution time. Further, they use OpenMP parallelization on the CPU of TX2 to parallelize the post-processing step.

On GPU, instead of 32b floating-point, they use 16b floating-point representation since going to lower bitwidth does not provide any further benefit. However, in some convolution layers, use of 16b format increases the execution time. This is because although 16b format reduces the number of cache accesses by allowing $2\times$ data to fit in the same cache line, it increases the number of executed instructions by up to $2.5\times$. Hence, they use 16b format in selected convolution layers only. On applying the above

optimizations, their system processes 20,000 images much before 10 minutes. To exploit the slack, they reduce the frequency of both CPU and GPU such that all the images can still be processed within the deadline with the least energy consumption.

Rallapalli et al. [32] evaluate the feasibility of running DNNs on embedded systems such as Jetson TK1. They note that due to its limited memory capacity and lack of memory management scheme, TK1 cannot run YOLO algorithm. They evaluate several techniques for improving memory usage efficiency. First, since the embedded system is used only for inference, no memory is allocated to variables that are not required during inference. This reduces the memory requirement of YOLO from 4.6GB to 2.8GB.

On TK1, CPU, which runs an OS, shares the memory with GPU. Here, there are two ways of performing memory management: (1) allocating memory on CPU and then performing a deep-copy to GPU by using `cudaMalloc()` and (2) using "managed memory allocation" such that only a single version of memory is allocated which can be accessed by both the physically-addressed GPU and virtually-addressed CPU. The limitation of approach (2) is that it incurs memory translation costs.

Since FC layers need highest amount of memory, they decrease the size of FC layer by reducing the number of filters in the last convolution layer and number of outputs of the first FC layer. After this, the network is re-trained. Further, while processing FC layers, weight matrix is read in steps to allow reuse of the memory and partial computation results are finally combined. The limitation of this strategy is that it increases the number of file accesses.

They also explore offloading the FC layers to CPU since CPU uses sophisticated memory management techniques. After offloading, GPU can start processing another frame, which leads to a pipelined architecture. Also, they increase GPU clock frequency to reduce the latency. Some operations take high latency when they are executed for the first time, e.g., for random number generation, one-time latency is incurred in initializing the state. They run these operations before executing the CNN to improve performance.

They show that their techniques increase inference speed significantly. Network-size reduction degrades accuracy but improves inference speed and saves energy. When the network size becomes small, the overheads of pipelining exceed its benefits. Also, reducing the network size beyond a threshold provides marginal decrease in inference time. Use of CUDA managed memory increases the inference time since it has to offer a unified view of memory to both the CPU and the GPU. Overall, their optimizations allow running YOLO on TK1 at nearly 4 FPS.

Otterness et al. [51] evaluate the impact of memory-management and co-scheduling policies on GPU of TX1. The GPU allows three memory management schemes. In "conventional scheme", the data need to be copied explicitly from CPU to GPU. This incurs data-transfer overheads but allows several optimizations. In "zero-copy scheme", the CPU and GPU access the same memory region which avoids the need of allocating GPU memory and data-transfer. However, this benefit is nullified by the fact that caching is not used for zero-copy memory. The "unified memory scheme" bears similarity with zero-copy in that in both schemes, memory pointers are shared, however, the unified memory scheme brings together the caching benefits of conventional scheme and easy programming benefits of zero-copy scheme. They note that with CUDA 8.0, conventional memory scheme provides comparable or better performance than the other two schemes for random and in-order accesses. Also, unified and zero-copy schemes provide similar performance with CUDA 8.0. With a different CUDA version (e.g., CUDA 7.0), the relative benefits of different schemes may vary and hence, choice of a proper CUDA version is important.

They further evaluate the impact of co-scheduling of applications on execution time. They note that WCET (worst-case execution time) with co-scheduling never exceeds the sum of the WCETs in isolation, but co-scheduling provides only minor reduction in the overall runtime. In fact, co-scheduling never leads to actual concurrent execution on the GPU and thus, it does not improve resource usage efficiency. Hence, any improvement provided by co-scheduling is entirely due to improved overlap between CPU-GPU computations or reduced latency in the GPU driver's kernel-scheduling queue.

As a case study, they execute two benchmarks on TX1, viz., traffic-sign recognition and CaffeNet, which is a single-GPU version of AlexNet. They note that when at most two instances of sign-recognition benchmark are executed, each of them achieves 30FPS. Similarly, with batch size of at most two, CaffeNet achieves 30FPS. However, on co-scheduling these two benchmarks, none of them achieves 30 FPS.

Luo et al. [91] present a technique for detecting and localizing robots. The previous techniques which detect robots based on its black color fail in case the color of the robot is changed. Their technique detects robots using RGB (red green blue) image and localizes them using the depth point cloud. They use Kinect-V2 as the vision sensor. The data provided by it is processed using TX2. In pre-processing stage, they transform and resize the input images. Pre-processing is accelerated using parallel execution on GPU. For robot detection, Tiny-YOLO is used with 9 convolution layers and 1 region-detection layer. They first register the "RGB image" to the "depth image" and generate the "depth-point cloud" on GPU. The "depth-to-color-mapping" information is stored in texture memory which improves performance compared to the use of global memory.

Since the size of each robot is same and spans a fixed range, they find the relation between box's size and distance between its center and the Kinect. From this, the number of pixels in robot-bounding box is estimated as a function of its distance from the Kinect. Based on this, the box that is unlikely to contain the robot is eliminated. This strategy is helpful in recovering loss of accuracy due to noise and blurring in images. Afterwards, the center of every box in the depth point cloud is searched to obtain the 3D position of each robot. Their technique detects and localizes the robots with high accuracy.

Yazdani et al. [45] note that out of the two steps of automatic speech recognition, viz., DNN-execution and Viterbi search, the latter becomes a bottleneck on CPU or GPU platforms. This is because Viterbi search traverses a large, irregular graph and hence, it shows unpredictable memory access pattern and poor data locality. Due to these reasons, it cannot be effectively parallelized and it leads to idling of compute resources. They propose running DNN on the GPU of Jetson and Viterbi search on their proposed dedicated accelerator. Both GPU and accelerator can access the same address space, so the GPU output can be read by the accelerator without the need of memory copy operations. As an additional optimization, the execution on GPU and accelerator can be pipelined.

3.2 Comparison of Jetson with other platforms

Table 4 shows the works that compare Jetson with related computing systems. Some works also compare the CPU and GPU of Jetson [12, 32, 45, 89]. We now summarize some of these works.

TABLE 4
A classification based on comparative evaluation of Jetson with other computing systems

Category	References
High-end GPU	[30, 31, 33, 34, 40, 47, 59, 73–75, 96, 98, 105, 106]
High-end CPU	[13, 29, 34, 53, 73, 107]
Raspberry Pi	[11, 13, 54, 55, 59, 70, 77]
FPGA	[12, 54, 55, 89]
Mobile-phones	Nexus 6P [13, 47, 96]
Others	Intel Joule [96], Intel Aero Drone [96], Intel UP [70], Intel Atom [44], Intel NUC [87], Intel FogNode [13], Intel Movidius neural compute stick [27, 106], Adapteva Parallella [12], TI DSP (digital signal processor) [12], Odroid XU4 [92], Samsung reconfigurable processor [92], NXP iMX6 Solo and Quad [107]

Hegde et al. [12] present a Caffe-compatible tool for generating and optimizing code for four embedded systems that fit within 20W power budget: a GPU (TX1), a DSP (TI Keystone II), an FPGA (Xilinx ZC706) and a RISC+NoC-based multicore (Adapteva Parallella). To efficiently map CNN algorithms to each platform, they evaluate hardware resources and constraints offered by each platform. For example, the DSP and FPGA perform pixel operations in 16b fixed-point format whereas GPU and Epiphany SoC support single-precision floating-point format.

For the GPU of TX1, they use cuDNN library which transforms convolutions into highly-parallel "single-instruction multiple-data" (SIMD) matrix-multiplication computations. These computations can be efficiently performed on GPU due to its ample amount of register file and SIMD resources [108]. These modules perform loop-unrolling which leads to a matrix K^2 larger than the original map, where $K \times K$ is the kernel size. However, on other platforms, direct convolution is more efficient than the matrix multiplication-style convolution due to their limited memory capacities and bandwidth. For other

platforms, they write code for various kernels and implement auto-tuning flows. They further discuss a range of optimizations for all the platforms.

As for benchmarks, they use five networks which, in increasing order of number of operations, are: small CNNs for MNIST, CIFAR10, STL10 datasets, AlexNet and a large CNN for Caltech101 dataset. They observe that for small networks, the runtime on GPU remains high due to lack of full utilization of its resources, and higher relative overhead of CUDA kernel launches. By contrast, DSP shows lower runtime since "very long instruction word" (VLIW) scheduling of ALU computations can be performed tightly, and direct memory access (DMA) traffic can be precisely controlled. For large networks such as AlexNet, GPU provides better performance since pooling layer cannot be efficiently implemented on DSP. The trends in energy efficiency are similar to that in performance.

The Parallela system shows competitive performance and energy efficiency (~4W power consumption) for MNIST and CIFAR10 benchmarks, since the intermediary maps can be saved completely in the scratchpad. However, for larger networks, the data need to be transferred to/from DRAM, which harms performance. The FPGA shows poor performance and energy efficiency for all networks due to limited peak capability of ALU and other architectural bottlenecks. GPU can be programmed most easily by using cuDNN, whereas DSP is the second best. The FPGA and Epiphany SoCs required more efforts to ensure correct operation, but allowed easy optimization.

Bechtel et al. [70] design "DeepPicar", a small-scale replica of NVIDIA's DAVE-2 (DARPA autonomous vehicle). DeepPicar uses the same CNN as used by DAVE-2 which has 250K parameters, 27 million connections and 9 layers. DeepPicar has 1:24 scale car, a webcam and a computing platform, which can be Raspberry Pi, Intel UP or TX2. CNN inference is performed on the images captured from the webcam and its output is the car steering angle at each control period.

They note that inference accounts for nearly 80% of the total execution time of the control loop. A single loop runs at 40 Hz on Raspberry (using all CPU cores but no GPU), ~80 Hz on Intel UP (using all CPU cores but no GPU) and above 100 Hz on either CPU or GPU of TX2. By comparison, the DAVE-2 ADV, which uses DRIVE PX, runs at 30 Hz. Thus, for the same, complex CNN, embedded systems give higher performance than the high-end DRIVE PX platform, which is surprising.

They further study the effect of co-scheduling on performance. They find that even on co-scheduling multiple CNNs, or bandwidth-intensive applications, TX2 gives real-time (upwards of 50 Hz and even 100 Hz) performance. For bandwidth-intensive application with write accesses, the slowdown on TX2 is much less than that on UP and Raspberry Pi. Even though the co-scheduled applications are not executed on GPU, still, TX2 shows slowdown due to sharing of memory system between GPU and CPU. Overall, their work confirms that modern embedded systems can run state-of-art machine-learning applications in real time, but shared resource contention can hamper their performance.

Biddulph et al. [87] evaluate the vision system for a humanoid robot capable of playing soccer. They compare TX2 with Intel NUC (model NUC7i7BNH). On NUC, "demosaicing" and "reprojection" modules are implemented in OpenCL, whereas on TX2, they are implemented in CUDA. Both these modules run in parallel and their output is fed to another module which performs "pedestrian detection" using SSD MobileNet network. Since OpenCL devices are not supported by TensorFlow, pedestrian detection is run on GPU of Jetson and CPU of NUC.

They note that for "reprojection" and "demosaicing" modules, the GPUs in Jetson and NUC provide nearly-equal performance. However, for executing DNN for "pedestrian detection", the CPU of NUC provides nearly three times performance than the GPU of TX2. Here, the execution time on TX2 includes the data-transfer latency, although this latency is not high. On NUC, pedestrian detection was performed on CPU and hence, its execution time need not include the data-transfer latency. The power consumption of NUC is 40.5W and that of TX2 is 9.5W. Overall, the NUC provides higher performance per watt than TX2.

4 Algorithmic optimizations to CNNs

The algorithm-level optimizations are vital for reducing the memory and computational demands of NN algorithms. Table 5 classifies the works based on the optimizations used in CNN algorithm/design. We now review works that reduce frame-count based on content-based filtering and sampling (Section

4.1) and reduce frame resolution (Section 4.2) and number of depth-map levels (Section 4.3). We then summarize works that use "knowledge distillation" approach to design a lightweight CNN (Section 4.4) or use an existing lightweight CNN (Section 4.5). Finally, we review strategies for finding the optimal CNN architecture for each input image and metric (Section 4.6).

TABLE 5
A classification based on algorithmic optimizations

Category	References
Lowering image resolution	[53, 57, 63, 71, 86]
Knowledge distillation	[30, 47, 97]
Transfer learning	[37, 41, 96]
Pointwise convolution	[46, 47, 75, 77, 80]
Tucker decomposition	[7, 72, 92]
Truncated singular value decomposition	[52]
CNN pruning	[32, 39, 56, 109]
Matrix-related optimizations	converting matrix-vector product into matrix-matrix product [38], splitting matrix-multiplication in FC layer [32], matrix tiling [39], exploiting matrix sparsity [39]
Other optimizations	merge batch normalization [92], loop unrolling [92], use of a-priori modules to remove most of the negative samples [75]
Determining the best DNN model to use	[65, 106]

4.1 Reducing the number of frames by filtering and sampling

Wang et al. [96] note that video-traffic from multiple drones can easily swamp the wireless spectrum. Also, strategies such as data compression [110] provide limited benefits since they are oblivious of the characteristics of the application running on drone, namely video processing or surveillance. They propose four strategies for reducing bandwidth consumption due to drone-data. These strategies can be applied separately or together. They assume that a drone communicates with an "edge-computing node", termed a cloudlet, which is located on the ground. On drone, processing is performed by a mobile device, e.g., TX2, or Intel's Joule or Nexus 6.

The first strategy, termed EarlyDiscard, performs content-based filtering to send only interesting frames. Given the limited compute-power of drone, they use lightweight DNNs such as MobileNet to filter-out uninteresting frames. This is because EarlyDiscard requires only image classification and not localizing the object in the frame. Since the existing MobileNet model is pretrained on images captured on ground, they use "transfer learning" [111] to finetune it over a small training set of images obtained from an aerial viewpoint. In transfer learning approach, a machine-learning model trained for one task is utilized as the starting point for a model on another task. Transfer learning improves accuracy for aerial images without requiring a large training dataset of aerial images.

The reduced dimension of objects in the aerial images interferes with the working of DNNs, since DNNs reduce the image-resolution before processing the images. Due to this, small yet interesting items may not remain distinguishable. To address this issue, they divide high-resolution frames into K (e.g., $K = 4$) sub-frames and then process these sub-frames. This division is done both during training and during inference on the drone and the cloudlet. Since the sub-frames are already lower in size compared to the frames, when the resolution-scaling is done on the sub-frames, their scaling factor is lower than that of a frame. Hence, the loss in accuracy due to resolution-scaling by DNN is lower with the sub-frame than with the frame. However, dividing a frame into K sub-frames increases the computation-load by K times.

They note that EarlyDiscard reduces bandwidth consumption significantly without harming accuracy or latency. The frame drop-rate can be increased if events happen rarely. The limitation of EarlyDiscard is that due to low accuracy of lightweight classifiers, it achieves low precision, i.e., it sends many false positives. To mitigate this issue, they perform sampling, whereby only a subset of frames are transmitted. Sampling can be performed alone or combined with DNN-based filtering to achieve large saving in the bandwidth. Sampling is useful when only simple object detection is required, however, when activity analysis needs to be done, sampling becomes ineffective since all frames having an object have to be

transmitted. Another limitation of EarlyDiscard approach is that by virtue of sending discontinuous frames, it reduces the efficacy of encoding techniques that leverage redundancy across consecutive frames.

The second strategy, termed "on-the-spot-learning" (OTSL) technique seeks to reduce false-positive frames transmitted by EarlyDiscard. Due to its higher resources, the cloudlet runs more powerful DNN to accurately detect true/false positives from the frames sent by the drone. Based on this, an OTSL filter is trained periodically and pushed to the drone. The OTSL filter functions as a cascade filter after the EarlyDiscard DNN and any frame flagged as uninteresting (negative) by OTSL filter is not transmitted. Overall, the OTSL filter learns features that confuse the EarlyDiscard DNN and are generally specific to the present flight, such as variations in terrain or object colors.

They use a linear SVM as the OTSL filter, which can be trained quickly with only a few training examples, has low memory footprint and inference latency. As for results, they note that the OTSL filter removes nearly 15% of the frames. However, in case of short or infrequent events, OTSL filter has fewer examples to learn from. Hence, it mistakenly removes true positive frames.

Drones with ample amount of storage, store the full-video locally to enable post-mission processing. In the third strategy, the cloudlet can fetch any frame filtered by previous two strategies, from the drone storage on-demand for completing the analysis. This improves event recall greatly with a minor impact on bandwidth utilization. The fourth strategy performs mission-specific optimizations such as detecting pixels with certain colors as life-jackets. Their technique saves bandwidth with negligible impact on output accuracy and latency.

Cavigelli et al. [104] present a technique for processing videos captured with static cameras where only few pixels change between different frames. For such videos, the conventional techniques process each frame of the video, which leads to wastage of resources without improving classification accuracy. Their technique first finds pixels for which the difference between successive inputs of any feature map/channel is higher than a threshold. Each such change impacts an area equal to the filter size and these output pixels are marked for an update. Then, instead of generating the entire image matrix, only those columns corresponding to the relevant output pixels are gathered. Their technique works on the observation that a change which is spatially localized at input is also so at the output and hence, number of pixels to be updated do not increase over different convolution layers.

The dimension of filter matrix is not changed and thus, the computation overhead of convolution is proportional to the number of changed pixels. Finally, the updated output feature maps are generated based on the results stored earlier, the recently obtained output values and the list of changed pixels. For further optimization, ReLU activation of modified pixels is also included in this step. Also, the effect of noise is attenuated after a few layers and this makes their technique robust to noise.

Their technique does not impact the training phase and can be used for an already-trained network. They evaluate their technique for an urban video dataset. Compared to the full-frame convolution performed in the baseline, their technique improves performance and energy efficiency significantly with minor loss in accuracy. A limitation of their technique is that it increases the memory requirement, although the total memory requirement still remains much lower than the memory capacity of TX1.

4.2 Reducing image resolution

A disparity map computes the horizontal displacement in each pixel between two images. Wang et al. [57] present a technique for disparity estimation which allows striking a balance between accuracy and speed. They first pass the input image pair via a feature extractor, which can compute feature maps at different resolution (e.g., scales 1/16, 1/8, 1/4). Their technique improves the accuracy of depth in multiple steps. In step one, 1/16 scale features are computed and passed through a disparity network to generate a low-resolution disparity map. Stage one has low latency due to the use of low resolution. In case more time is available, their technique enters step 2 where features of 1/8 scale are obtained. In this step, only a correction of the map from step one is computed, since these errors can be detected at higher resolution. If even more time is available, step three is executed which is similar to step two, except that the scale is 1/4 which doubles the resolution. In step four, the map of step 3 is refined using a spatial propagation model [112].

The major advantage of their technique is that it computes the entire disparity map only in step one at very-low resolution. In other steps, only a residual is computed. Depth estimation latency increases

cubically with the resolution and linearly with highest disparity (D). At high resolution, value of D between two pixels can be very large (e.g., 192), but on computing correction of existing disparities, D is much smaller (e.g., 5), and due to this, their technique achieves large speedup.

Results: At any point of time, their technique provides the present estimate of the depth map. With increasing amount of available time, their technique generates disparity maps of higher accuracy, and the map produced from step four has very high accuracy. Their technique reduces the number of parameters by orders of magnitude and on TX2, achieves a frame rate between 10 to 35.

4.3 Reducing number of depth-map levels

Tsai et al. [53] propose a technique for detecting obstacles using depth map generated by the stereo camera and the distance estimated by "monocular camera" model. Pixels with same disparity are likely to belong to the same object, which is assumed to be an object on the road. Based on this, their technique searches for pixels with same disparity on the depth map in the vertical direction. Use of depth values from stereo vision removes the inaccuracy in distance estimation from monocular camera.

They use a stereo camera for obtaining disparity map and find the distance in the map using triangulation. Their stereo camera can provide output resolution of 3840x1080 (i.e., 1080p), 2560x720 (i.e., 720p) and 1344x376. Use of higher resolution allows estimating larger distance range, but also increases the computation latency. Further, their stereo camera provides three depth-modes, viz., "quality", "medium" and "performance" which incur decreasing amount of latency in generating the depth-map. To balance various factors and achieve high frame-rate, they use 720p resolution with "performance" mode.

Due to limited resolution, their technique detects obstacles in 80m distance range. If the depth map has 256 levels, each level covers distance range of 0.31m (=80m/256). This range is too small to fully enclose typical objects in a single level and makes range-partitioning sensitive to the jitter in the camera due to driving vibration. Hence, they use only 16 levels in the depth map which also increases the continuity of the objects. Thus, each level covers a range of 5m (=80m/16). They note that use of 16 levels provides higher accuracy than using 8 or 64 levels.

Their technique seeks to classify the image pixels into "road" and "obstacle" categories. Results show that their technique achieves high accuracy. For obstacle-detection in each image, their technique takes only 4ms and 16ms latency on Core i7 3.6 GHz processor and TX1, respectively. Their technique achieves real-time performance even on embedded system due to use of fewer depth-map levels, simple search operation and use of monocular camera model with stable parameters. Further, if majority of obstacles are near-by, they are localized in the first depth-level itself, which reduces the processing time. Compared to 2D image object recognition based techniques, their stereo-vision based technique has higher accuracy and lower latency. Also, stereo camera is cheaper than a LIDAR or RADAR. The limitation of their technique is that it cannot accurately detect very short obstacles such as those with less than 10 cm length.

4.4 Reducing CNN size using knowledge distillation

In knowledge distillation approach [113], a small network, termed "student network" is trained to imitate the output of an original larger network, termed "teacher network". In effect, training transfers the knowledge of the teacher network to the student network.

Reddy et al. [30] present a CNN-based "driver-drowsiness detection" technique. It classifies an input image of driver's face into one of the three categories: "normal" (no fatigue), "yawning" (likely to become drowsy very soon) and "drowsy". Their technique works in two phases. In the first phase, "face detection and alignment" are performed using "multi-task cascaded convolutional networks" [114]. This provides coordinates of face boundary and locations of five items: left/right-eye, nose, left/right-lip-end. These values are provided to the second phase which detects the drowsiness. For this, they propose three models. (1) "Four-stream model" which takes 4 streams as input: left/right eye, face and mouth. Each stream is processed using a CNN with 5 convolution layers and one or more FC layers. These FC layers finally connect to two FC layers.

(2) Since use of 4 streams in the "four-stream" model makes it bulky and slow, they propose a "two-stream" model which uses only 2 streams, viz., left-eye and mouth. This is based on the following observations (A) for face recognition, incomplete facial images can provide features comparable with the

full image. (B) Due to the symmetry of eye-movement, using the image of only one eye provides similar accuracy as using the image of both the eyes. (C) The image of mouth is crucial in detecting yawning stage. They confirm that using only 2 streams in the "four-stream" model leads to negligible loss in accuracy. Yet, the two-streams model reduces the latency to half compared to four-streams design.

(3) To reduce the latency even further, they use "knowledge distillation" technique [113]. This technique uses a "teacher network" and a "student network". The teacher network is the original large network which is trained from the dataset itself. The student network is a smaller and faster network and it learns features from the "soft targets" generated by the teacher network. In their design, "slimmed-two-stream" network works as the student network whereas the stream-two works as the teacher network. Both two-stream and slimmed-two-stream networks use the same filter sizes, but the slimmed-two-stream network uses smaller number of kernels in convolution layers.

The results obtained on TK1 using their custom dataset are summarized in Table 6. Evidently, while the two-streams model is most accurate, the slimmed-two-stream model is the fastest. Also, their proposed techniques outperform the "faster-RCNN" [115] based on AlexNet architecture. They also note that "faster-RCNN" based on VGG16 architecture cannot be deployed on TK1 since it takes more than 2GB memory.

TABLE 6
Results of Reddy et al. [30]

	Model size (MB)		Accuracy	End-to-end speed (FPS)	
	On disk	During execution		On GTX 1080	On TK1
Four-streams	56	600	91.30%	72	6.1
Two-streams	28	443	93.80%	82	12.5
Slimmed-two-stream	10	353	89.50%	90	14.9
Faster RCNN (AlexNet)	236	845	82.80%	22.7	1.1

Zeng et al. [47] present a technique for recognizing a pill from its image. Here, a user takes image of a pill from his mobile phone. Then, their technique searches a database of high-quality images and finds a list of most similar pills to the query image. Due to factors such as illumination, phone orientation and existence of a large variety of pills, achieving high accuracy is challenging.

Since the database images have uniform background and plain texture, the pill can be localized and segmented using "gradient detection" technique. For query images, however, this simple technique does not work well, since these images are taken in disparate backgrounds. Hence, they train a pill detection engine using SVM. They further apply "Gaussian filtering" for simulating blurred images, zoom the images to ensure robustness to changes in pill-size and apply random translation and rotation to ensure robustness to localization errors.

For achieving invariance to loss in image quality due to noise, they use "triplet loss function". Further, they use three independent CNNs: "color CNN", "gray CNN" and "gradient CNN" for obtaining color, shape and imprint details of the pills. These CNNs are based on AlexNet. For a query image, their technique measures its similarity with standard image of each pill-category in the CNN feature space. For this, the similarity scores of each of the three CNNs are added together.

A limitation of the above multi-CNN architecture is that it has huge memory and computational demands. To reduce these, they use "knowledge distillation" strategy to train a smaller student network and further apply many optimization strategies. Their technique achieved first prize in the national library of medicine "pill image recognition challenge" and use of multiple CNNs provides much higher accuracy than using only one CNN. Although knowledge distillation strategies reduce the runtime, this decrease is not in proportion to the decrease in FLOPS, especially on GPU. This is due to the parallel execution on GPU. These strategies also reduce memory utilization at runtime.

They compare execution on TX1 with that on a desktop with i7-5930k CPU and GTX 1080 GPU and "Samsung Galaxy S7" CPU. For the network with smallest model size, the runtimes are as follows: 114ms on desktop CPU, 3.8ms on desktop GPU, 19ms on TX1 GPU and 1335ms on S7 CPU. Further, TX1 consumes lower energy than S7 smartphone and the largest contributor to energy is CNN feature extraction. Although the power consumption of teacher and student networks are similar, the student network consumes much less energy due to its smaller runtime.

4.5 Using lightweight CNNs

CNNs designed for complex tasks have high memory and computational demands. Hence, they do not provide satisfactory performance on low-resource systems. To mitigate this issue, researchers have designed lightweight CNNs that tradeoff accuracy for reducing CNN latency and memory demands. For example, tiny YOLO-V2 has two times smaller number of layers compared to YOLO-V2. On COCO dataset, the mean average precision of tiny YOLO-V2 [116] is nearly half of that of YOLO-V2 [116], yet, the tiny YOLO-V2 has nearly 12× less computations and 6× higher FPS compared to YOLO-V2. We now review works that design and/or deploy lightweight networks for achieving high performance on embedded systems such as Jetson.

Sadique et al. [63] present a technique for identifying faces of suspected people from CCTV footage. They design a 27-layer reduced ResNet, which brings significant reduction in the number of computations compared to 34-layer and 50-layer versions of ResNet network [117]. Also, compared to 34-layer ResNet [117], their 27-layer Reduced-ResNet reduces the number of filters per layer by one-half and the dimension of pooling layer is reduced from 1000 dimensions to 128 dimensions softmax. Further, the training of Reduced-ResNet begins with random weights and "cross entropy loss" is used which maps all target individuals to mutually exclusive regions. These optimizations allow their technique to run on TX2.

Their technique first reduces 640×480 sized images obtained from the video to 128×128 faces. Then, each face is encoded as a 128-bit vector. They note that minor perturbations to even few features of the face can confuse traditional image classifiers and this weakness can be exploited by a suspected person who can change his/her facial appearance to evade detection. To mitigate this issue, they use "generative adversarial networks", which are effective in generating realistic adversarial images, that can be used for robust training of the facial recognition system. In "generative adversarial networks", the generator produces the data and the discriminator utilizes the same data for ascertaining authenticity of the input. They use "adversarial loss" for making the generated images identical to the real images. The adversarial images are similarly encoded and fused with the deep feature comparator for detecting the suspects. On TX2, their technique achieves 99.4% accuracy in finding whether two facial images are from the same person, while consuming only 17.6 Watt-hour power.

Jung et al. [81] present a technique for recognizing gates, so that an autonomous drone can fly through them. The traditional techniques based on color and gate-size do not work well since multiple gates situated behind each other appear to overlap. In their technique, the drone uses a LIDAR for measuring the altitude, an optical flow camera for measuring ground speed and a stereo camera for measuring odometry. The odometry and velocity data are fed to the "unscented Kalman filter" which fuses above information with IMU (inertial measurement unit) sensor to obtain more accurate value of velocity and position. All the processing, including execution of unscented Kalman filter, happens on TX2.

Their technique uses SSD [5] for detecting the gate. Since SSD incurs high latency, they replace its base network from VGG-16 to AlexNet and remove one FC layer. Since only one gate needs to be detected, a few non-essential high-level feature layers are also removed. Further, the regression operation for finding small objects was alleviated since the distance between gates is at most five meters. To recover the loss in precision due to above optimizations, they finetune the network parameters. Specifically, the box sizes used in SSD were ascertained based on their dataset, since the original SSD is trained on PASCAL VOC dataset. For example, since gate-size is not small, the box size in SSD was increased. On detecting multiple gates in the field of view, their technique selects the closest gate since it has highest probability.

On detecting a gate, their technique extracts its center point and runs the "line-of-sight guidance" algorithm for making the drone follow the center point. The guidance algorithm determines forward/left/right movement and linear velocity. The algorithm seeks to minimize the lateral error between the drone and gate center. Experiments confirm that their drone passes through all the nine gates with near-zero lateral error and can also pass through a moving gate.

Ghazi et al. [73] present a DNN-based technique for smile detection. Their technique first detects face using Viola-Jones algorithm [118]. Then, face alignment is performed which removes issues due to rotation and scale and ensures that facial parts are at a fixed location. Their face alignment technique uses an ensemble of regression trees to find the location of a fixed number of face landmarks and matches them to a landmark database. As for DNN, they experiment with six architectures: Inception V3, VGG-16,

Inception-ResNet-V2, ResNet, XCeption and MobileNet with different values of "feature map multiplier" and "resolution multiplier". Different tasks such as obtaining the frame, detecting face and smile are performed by different threads and they work in pipelined manner.

As for results, they note that VGG-16 and Inception-ResNet-V2 do not run on TX2 due to memory size issues. MobileNet provides highest frame-rate for all its parameter-settings. Further, by changing its parameters, its accuracy can be altered such that its highest accuracy is close to that of XCeption, ResNet-50 and Inception V3. Overall, although their proposed model achieves slightly lower accuracy than SmileNet [119], their model can run on an embedded platform whereas SmileNet cannot run on an embedded platform.

Carrio et al. [94] present a technique for detecting drones using depth maps. This is based on the observation that depth of a flying object is different from that of its background and thus, a flying object leads to discontinuity in the depth-map which provides a visual clue for drone detection.

In their technique, a DNN takes a depth-map as input and predicts the bounding boxes having a drone along with a confidence value for each bounding box. Due to the noise in the stereo matching process, the bounding boxes predicted by the model may not indicate the true location of the drone in the depth image. To avoid these errors, they ascertain the location of the drone as the 2D point with least depth in the bounding box. This is because the object to be detected (drone) should be closer to the camera than the background. The 2D point chosen above is reprojected to 3D for obtaining the actual drone relative position.

Image acquisition and processing is performed on TX2. As for DNN, they run Tiny-YOLO-V2 for drone detection with 672x672 images since high-resolution images facilitate detection of smaller or distant objects. They set drone flying speed at 2 m/s. They observe that processing a frame takes 200ms and thus, a drone can be detected every 0.4m distance which suffices for avoiding collision. The frame-processing latency is high since the GPU in Jetson is used at the same time by camera for stereo matching and by the deep neural network for performing detection. Their technique can perform detection with high accuracy up to a distance of 9.5m.

Cao et al. [80] note that many CNNs perform detection in last layer only but since the last layer effectively uses bigger receptive field in the image, it cannot detect small objects precisely. They present a CNN architecture for detecting small objects, e.g., pedestrians, cyclists or distant cars. Their CNN architecture is modular and consists of two modules, termed front module and smaller module. The front module has three 3×3 convolution layers connected to a 2×2 max-pooling layer. The stride of first convolution layer is 2 and that of remaining layers is 1. These convolution layers use large feature maps since the downsampling is performed towards the end of front module. By virtue of this, front module decreases the loss of information from input images. The smaller module has two 1×1 convolution layers and two 3×3 layers. 1×1 layer performs pointwise convolution which reduces model size, increases non-linearity of decision function and retains accuracy. Their final CNN architecture stacks the two modules in different manners for fusing contextual information at different-scales. Their technique detects small objects with high precision and achieves 100FPS on TX2.

4.6 Choosing the optimal CNN for a metric

Taylor et al. [65] note that different DNNs (e.g., MobileNet, ResNet, Inception-V2) provide lowest inference time or highest accuracy on different metrics (top-1 or top-5) for different images. They present a technique for selecting the best DNN in terms of latency and accuracy. They use a "model-selector" which selects the best DNN to use for each input image. The selector works based on image features such as its brightness and edges. For the selector, they experiment with four classifiers: decision trees, support vector machine, CNN and k-nearest neighbor (kNN). Of these, they select kNN due to its high accuracy and low (<1ms) prediction latency.

If there are Q DNNs to choose from, then the selector uses Q kNN models connected in series. The ith kNN predicts whether the ith DNN can be used for classifying an input image. The DNNs are ordered such that the first DNN is the one that is optimal for most of the training data. For the remaining data, a DNN is chosen which is optimal for most of the training data and this is used as the second DNN, and so on. This process is terminated when adding a DNN improves the accuracy by less than a threshold

and the number of DNNs added till now is the value of Q. The selector is trained offline using supervised learning to find the optimum classifier for every image based on its features. They use 30 features such as contrast, edge, aspect ratio, etc. Compared to the best single model, using their technique with $Q = 3$ improves accuracy and reduces the latency of inference.

5 Application areas

Table 7 classifies the works based on their application domain. These areas are, by no means, exhaustive since embedded machine learning is poised to transform many more important fields. Table 7 also shows some of the competitions along with the research papers that discuss their entries submitted to those competitions.

TABLE 7
A classification based on application domain

Medical	Stress detection [54], seizure detection [37], diabetic foot ulcer detection [98], diabetic retinopathy detection [33], fall detection [46]
Farming	weed detection from drone images [58], in-trap pest detection [77]
Speech recognition	[38, 45]
Robotics	soccer-playing robot [87, 91], ball and goal detection in soccer [43], tennis ball detection and collection [49], underwater-swimming [69], object grasping [76], robot locomotion/movement [44, 61], service robot for providing navigational assistance to visually impaired [50],
Image processing	semantic segmentation [60], face detection [63, 95], smile recognition [73], hand gesture classification [64], image classification [11, 65, 67], scene labeling [40], depth estimation and drone-detection from stereo images [42, 57, 94], object detection [52, 67], railway traffic detection [75], object localization [91, 97], multimodal data classification [89], key-frame extraction [78, 79]
Autonomous driving	obstacle/vehicle detection [36, 53, 66, 68, 80, 107], pedestrian detection [31, 36, 80, 87, 107], curb and drivable area detection [68], driving lane detection [105, 107], parking lane departure detection [74], driver drowsiness detection [30],
Drone	object/vehicle detection from drone images [24, 72, 83, 93, 99], trail navigation [41], passing a drone through a gate [81], wire detection [62], autonomous landing of a drone on a moving ground vehicle [82]
Well-known competitions	LPIRC [7, 52], design automation conference (DAC) system design contest [72], RoboCup [43, 91], autonomous drone racing [81], AI City Challenge [67, 79], national library of medicine pill image recognition challenge [47]

In the remainder of this section, we summarize some works in the area of medical and farming (Section 5.1), robotics (Section 5.2), autonomous driving (Section 5.3) and drone navigation (Section 5.4).

5.1 Medical and farming

Sa et al. [58] present a technique for classifying weeds based on multispectral images obtained from a drone. Accurate weed detection minimizes harm to useful plants and thus, is useful in autonomous farming. They use a drone with downward-facing camera which flies over three different areas containing (1) only crop, (2) only weed and (3) both weed and crop. For cases (1) and (2), ground truth is generated by finding the vegetation cover and for case (3), manual labeling is used. Semantic segmentation is performed using SegNet [120] which is an "encoder-decoder cascaded CNN". A pixel is classified as belonging to one of the three classes: background, weed or crop. For improved class balancing, the frequency of appearance of each class is modified according to the training dataset. The drone collects images at 1Hz and transmits them to a ground station. They show that despite the limited dataset and challenges involved, their technique provides more than 80% accuracy. Further, although Titan X GPU has 3.6× lower latency than TX2, its power consumption is 17.8× higher. Further, on both these systems, inference time is not affected by the number of input channels.

Goyal et al. [98] present a technique for detecting and localizing "diabetic foot ulcers". They first create a large diabetic foot ulcer dataset with 1700+ images where the ground-truth has been annotated by two experts in the form of bounding box of diabetic foot ulcer. Their algorithm first uses a CNN for extracting the features from input images. Based on these, the regions containing the ulcer are identified. Finally, these regions of interest are classified and their locations are further refined. For each of these steps, they

evaluate various well-known CNN architectures such as Inception-V2, MobilleNet, ResNet, Faster R-CNN, etc. For example, the SSD-MobileNet architecture uses MobileNet [121] for classification and SSD [5] for localization. It is a lightweight architecture with a model size of only 29MB.

Since medical dataset is very limited, they use both partial and full transfer learning approach to transfer the features from a model trained on other datasets such as ImageNet to their domain. They note that each inference with SSD-MobileNet architecture takes 30ms on Titan X GPU and 70ms on TX2. Further, using the onboard camera, visualization of diabetic foot ulcer can be performed at 5 FPS using SSD-MobileNet architecture.

Ardiyanto et al. [33] present a system for helping the doctor in assessing the severity of the "diabetic retinopathy" from the images of retina. They use a "cascaded encoder-classifier network" which achieves low model size due to its residual-style architecture. The encoder has five cascaded convolution blocks. The classification stage performs log-softmax operation to find whether the encoder has produced strong features. The network classifies an image into multiple grades of severity of diabetic retinopathy, such as no, mild and severe diabetic retinopathy.

The model sizes of ResNet-20 and their network are 42MB and 7.8MB, respectively. For each forward inference operation, ResNet-20 and their network take 37.3ms and 3.8ms, respectively on GTX 1080. On TK1, their network takes 41ms for each inference but there is a small loss in accuracy. ResNet-20 cannot run on TK1. Overall, their work shows the ability of TK1 in assessing the severity of diabetic retinopathy directly from original images without any human intervention.

Page et al. [37] present a technique for detecting epileptic seizures using scalp-based EEG (electroencephalogram). They use a CNN with max-pooling layers for performing feature extraction and classification. They evaluate three models: (1) a generic model which does not use any patient-specific data (2) a patient-specific model and (3) a compound model which fine-tunes a generic model with patient-specific data. They show that on all metrics, viz., latency, detection accuracy and number of false alarms, the compound model is superior to the patient-specific model, which, in turn is better than the generic model. Lesser effectiveness of generic model is due to use of data from limited number of patients.

They implement their technique on TK1 and experiment with different number of CPU cores, using or not using GPU, and using different frequency for CPU and GPU. Their system works in two phases: an idle phase where the system waits for sufficient data to be read and an active phase where classification is performed. They show that use of GPU lowers execution time which improves battery life by allowing the system to stay in idle phase for larger duration. On configuring GPU at 72 MHz and CPU at 2.32 GHz, their technique classifies a 15-second segment in 0.3 ms and allows a 5400 mAh battery to last for 81 hours.

5.2 Robot navigation

Pierre et al. [85] present a technique for performing perception and visuomotor control to allow a robot to follow another robot. Their technique uses a spatio-temporal DNN and takes only camera images as the input. Their end-to-end deep-learning approach for visuomotor control trains a single DNN for directly mapping camera feed to motion controller decisions. Thus, they do not use intermediate modules for tasks such as segmentation, feature-extraction, object detection, localization or path-planning.

For controlling linear and angular velocities, the steering and throttle/brake (respectively) need to be controlled. For this, they utilize a multi-branch DNN with shared and task-specific layers. Training of DNN is performed with a "multi-task learning scheme" using a dynamically-weighted joint loss function.

Their technique perceives robot's own motion by studying its environment and relative velocities of other objects. Spatio-temporal DNNs leverage time-series of images for improving perception of scene kinematics. Further, the memory capacity present in DNNs allows the robot to follow the leader even if the leader has moved outside its vision for short time-periods.

They experiment with five architectures: a baseline architecture which performs spatial reasoning at every time-step and four other architectures that analyze frames across multiple time-steps. (1) With baseline CNN architecture, visual spatial features e.g., objects and their position, depth and pose can be perceived. Then, braking control can be found from the leader's position in the vertical field of view and steering angle can be determined from leader's pose and horizontal position. (2) The stacked CNN architecture adds more input channels to the baseline architecture. This allows handling multiple video

frames in addition to pixel RGB values. (3) In 3D CNN architecture, 3D kernels perform convolutions with video frames that are seen as data cube with two spatial and one temporal dimension. Thus, time-information of input is preserved in all the convolution layers which allows the CNN to perceive structures and patterns at different levels of granularity.

(4) RNNs are capable of processing input sequences of any length, and their memory allows recording past information. Their RNN-architecture is designed by replacing the FC layer of baseline CNN with an RNN layer composed of "gated recurrent unit cells". (5) The attention-based scheme allows learning which temporal connections are important across feature vectors at various time steps. Use of multiple attention-heads facilitates perceiving different temporal features in the same architecture. In this architecture, instead of using RNN, they use multi-headed self-attention scheme after the FC layer.

The experiments are performed using both simulation and real-world experiments. The simulation results show that the RNN and 3D CNN have the highest mean-time-to-failure. Also, RNN and attention-architecture are the best in throttle and steering accuracy. The RNN model also leads to least acceleration and jerk, which means that it provides the most smooth driving experience. This is because RNN performs explicit modeling of flow of time instead of learning direct feature relationships between frames separated far apart in time. The baseline architecture has the least number of parameters and training/inference time. Among others, stacked DNN has the least and 3D CNN has the highest number of parameters. On increasing the length of video clip, the parameters of 3D CNN also increase whereas that of other architectures remain (nearly) same.

In the real-world experiments, two similar robots were used as leader and follower and were fitted with TK1. These experiments confirmed effectiveness of their technique and showed the latency of 200ms. Also, their follower robot tracks the leader for long time-period (e.g., 30min) without hitting the obstacles or departing from the lane even in presence of intersections and curves.

Manderson et al. [69] present a vision-based controller for "AQUA robots" that can perform underwater swimming. An AQUA robot is a small hexapod with six degrees of freedom. Their controller-algorithm guides the robot to swim close to the coral reefs without getting into collisions and also avoiding coral-free areas. The CNN is based on ResNet-18 architecture and it decides the relative steering angles based on the image data. Its decisions are used to change the orientation of AQUA robot. The control algorithms are run by other computing systems such as an i3 processor, whereas neural networks are run on TX2. On TX2, their algorithm processes images at 10 Hz and has an accuracy of 41% for classification into 7 categories.

Gu et al. [49] present a "tennis ball collection" robot. It uses YOLO algorithm to detect tennis balls. Then, it it performs path-planning to collect the ball and for this, it uses "pointer networks" algorithm [122] to solve the "travelling salesman problem". Both the algorithms are implemented on TX1. Results show that their technique recognizes the ball accurately and efficiently performs path-planning to pick it up.

5.3 Autonomous driving and traffic surveillance

Wang et al. [105] present an algorithm for lane detection which works in two steps. In first step, each pixel is classified as to whether it belongs to a lane and this creates a lane-edge estimate. In second step, lane lines are localized based on lane-edge estimates. A separate DNN is used for each step. For first step, an encoder-decoder design is used where "stacked depthwise separable convolution" and 1x1 convolution layers are used for encoding the features and "non-parametric decoding layers" for recovering feature resolution. The estimated map is converted to lane-edge coordinates and is supplied to the second step. The DNN for second step consists of a "point-feature encoder" and an "LSTM decoder" and it localizes the lane-lines accurately. They note that on Titan XP GPU, the first and second steps run at 330 FPS and 1300 FPS, respectively and thus, the overall algorithm runs at 250 FPS. On TX1, a speed of 26 FPS is obtained which suffices for real-time operation. The model size of whole algorithm is below 1GB.

Borghi et al. [86] present a technique for head pose estimation, which is useful for several purposes, e.g., analyzing the behavior and attention of driver. They use an RNN with 5 convolution layers, 3 max-pooling layers and 2 LSTM modules. The RNN works in regression manner to output continuous values of the three "3D Euler head angles", viz., roll, pitch and yaw. The RNN works only on depth images,

obtained from an infrared sensor. They crop the image of the face such that only a small part of the background is included in the image. They use "data-augmentation" to avoid over-fitting and to create partially occluded images so that their technique can be robust against head occlusions. Their technique achieves good accuracy, except for frames with occlusions.

The small size of images (64x64 pixels) and shallow architecture of RNN allows them to achieve high frame rate on TK1 and TX1. Specifically, on TK1, frame-rate with CPU and GPU are 10 and 19 FPS respectively, and on TX1, it is 15 and 30 FPS, respectively. TX1 provides higher performance than TK1 because it has higher number of CUDA cores, more powerful processor and it supports cuDNN libraries.

Bura et al. [90] present a technique for helping drivers in finding a vacant parking location and charging them based on the exact usage of parking space. They use multiple "ground" cameras which are connected by WiFi. Further, multiple connected "top-view" cameras are used that have wide-angle lenses to cover the whole parking lot. From their feeds, the occupancy of all parking slots is detected and tracking of vehicles is done till they get parked.

Both types of cameras interface with Raspberry Pis that are used for recognizing license plate numbers and tracking vehicles to log their entry/exit times. The top-view cameras are also interfaced with a Raspberry Pi which feeds its recording to TX2 with a latency of 10 seconds. TX2 tracks a vehicle till it is parked in a parking lot. For tracking, Tiny-YOLO is used and for classifying a parking-lot into empty or occupied state, a lightweight NN with only one convolution layer, one ReLU, one max-pooling and three FC layers is used. For binary-classification, this NN provides accuracy close to AlexNet but has only one third latency. The data about vehicles and parking-slots is sent to a central server which can be queried to obtain information such as free parking lots. Their technique achieves high accuracy and reduces the number of cameras required for achieving desired level of service quality.

Sreekumar et al. [79] present a technique for collecting and analyzing traffic pattern at intersections. They place the camera at a traffic intersection. Based on the width of each-lane, number of lanes on each side and vehicle speed, they find that capturing 6 frames every second is sufficient to track a vehicle. They manually mark the lanes on the field of view. For object detection, they use YOLO-V2 since it gives higher precision than Faster-RCNN. The bounding boxes found by YOLO-V2 are passed to the object tracking module which detects the amount of change in the scene. For finding the movement in each object, edge detection is performed on the difference between frames. Then, contours of detected edges are obtained and motion is represented in the form of rectangles. Finally, motion rectangles are localized using clustering approach.

For object tracking, they use four algorithms from which one algorithm is dynamically selected depending on whether the object movement across frames was large-enough and whether the target object was partially/fully occluded. The order in which the algorithms are used depends on their speed and ability to discern tracking failure. Their movement learning engine tracks each bounding box and learns its key characteristics such as direction, speed, type of object, location of objects in the field of view. Finally, based on the coordinates of the lanes meeting at the intersection, information about vehicles entering and exiting a lane can be found. This data is useful for making traffic-management decisions. Their technique achieves 3FPS when running on TX2. By extracting key-frames, the frame-rate of their technique can be further improved.

5.4 Drone navigation

Sanket et al. [71] present a bio-inspired approach which allows a quadrotor drone to fly through a gap using only a monocular camera and onboard sensors. Their technique does not require developing an explicit 3D model of the scene. Their technique finds the contour of the opening as the position where the discrepancy in spatial depth is maximum. The pixels with higher and lower depth values are assumed to be part of background and foreground, respectively. The gap is found using "temporally stacked spatial parallax" (TS^2P) algorithm. Optical flow between frames is computed using FlowNet2 [100] which provides high accuracy and has acceptable latency. Their control scheme for flying is based on "contour visual servoing". As the drone flies, their technique tracks the contour location using "label set propagation" algorithm.

Their technique controls the altitude and position of drone to fly inside the gap. For this, it uses a PID (proportional-integral-derivative) controller which aligns the projection of the drone center on the

image plane with the safest point. Note that the safest point is the mean of the safe region. They perform experiments using a drone which is fitted with a forward facing camera, TX2, and an optical flow sensor coupled with a sonar. The vision and control algorithms run on TX2. Their technique works well for different shaped gaps including narrow gaps with only 5cm tolerance. Also, it can handle dynamic noise and works with even small drones that can host a camera with resolution of only 32*48 pixels.

Detection of wires is useful for drone navigation. However, wire detection is challenging since wires are very narrow, distinguishing them from looking-alike lines and edges is difficult and they may be present in any location or orientation. To mitigate these challenges, working on high-resolution images is essential. Also, neural-network architectures that perform up/down-sampling are not suitable for their task since they lead to loss of information as the wire-width is only few pixels.

Madaan et al. [62] present a technique for wire detection which uses only monocular camera for perception. They use dilated convolutional layers which allow gathering context without decreasing feature map size. Dilating a layer by d is same as having $d-1$ alternating zeros between the learnt elements. To further improve the efficacy of this model, "context module" can be added to the segmentation networks. Context module has seven layers with increasing dilation factors. They generate 30 architectures by changing the number of channels, pooling layers and dilation factors in front-end module and number of layers and dilation factor in context-module. From these, they select five architectures that provide highest accuracy and/or least inference time on TX2.

They generate synthetic dataset by generating wire-images and superimposing them on the frames obtained from the video. Since wire pixels are much less in number than the background pixels, they account for this imbalance in the loss function. To also provide a local evidence of the presence of the wire, they append the results of line detection and edge detection to CNN's input in the form of two extra channels.

Results: they observe that architectures with more dilation provide higher average precision. With increasing number of layers, inference time also increases. Providing the results of line and edge detection improves precision in synthetic dataset but reduces precision marginally in real dataset. On TX2, their top five architectures provide FPS in the range of 2.8 to 4.4. Overall, their architectures provide higher precision and/or speed than previous techniques.

6 Conclusion and Future Outlook

In this paper, we presented a survey of techniques for accelerating neural-network applications on Jetson platform. We reviewed the key features and challenges of Jetson vis-a-vis other computing platforms. We discussed both hardware-level and CNN-level optimizations and showed how Jetson is accelerating cognitive tasks in a wide-range of application areas. This survey will be useful for researchers and practitioners in the area of artificial intelligence, chip-design, embedded system and computer architecture. We close this paper with a brief discussion of future challenges.

Zhang et al. [13] note that while running PyTorch on TX2, loading AlexNet model takes 700 times the latency of inference. Thus, designers of Jetson should also focus on reducing the model-load latency which will especially benefit short-duration tasks. Zhang et al. also note that different software packages such as Caffe, TensorFlow, PyTorch etc. may lead to different values of latency, memory footprint and energy consumption on running a neural network model. Given this, a careful choice of software framework, compilation toolchain, etc. is required to harness the best performance from an ML accelerator such as Jetson.

As researchers continue to propose novel machine learning architectures for increasingly complex cognitive applications, the resources and efficiency provided by even most recent computing platform appear tiny in no time. Jetson is no exception to this rule. Going forward, improvements are required in all layers of system-stack. At circuit-level, moving to smaller feature sizes and use of non-volatile memories can allow high integration density with near-zero idle power. At microarchitecture level, closer integration of CPU and GPU can reduce data-transfer overheads. Also, customization of GPU architecture for machine-learning workloads such as inherently supporting binarized neural-networks can provide even higher efficiency. Jetson TK1/TX1/TX2 platforms do not support INT8 operations which limits their ability to perform low-precision deep-learning.

Similarly, while the ARM CPUs used in Jetson consume very small amount of power, their performance is generally lower than that of Intel Atom CPUs. Due to this, the application phases executed on CPU, such as pre/post-processing operations, may become a bottleneck on Jetson. Low CPU performance also limits the efficacy of CPU-GPU workload partitioning techniques. To deal with this issue, an improvement in performance of CPUs is also required. At architecture-level, techniques such as dynamic power-gating and frequency-scaling can be helpful in saving energy [123]. Also, since data-movement energy accounts for a large fraction of the overall energy [124], efficient encoding techniques are required for reducing these overheads.

At algorithm-level, leveraging the error-tolerant nature of applications and humans will allow trading-off minor loss in accuracy with disproportionate gains in efficiency. Since code written for high-end processors does not provide optimal performance on an embedded system [107], effective compiler techniques are required to reduce code-rewriting efforts and still match the architectural characteristics of Jetson. Further, OS techniques can be helpful in transforming memory-layout and access pattern for removing the memory bottleneck. Even though the application areas of Jetson are impressive, most of these applications are related to computer-vision only. There are other key areas such as natural language processing, finance, defense, aerospace, health, etc., where embedded deep learning can be highly effective. Moving forward, deep-learning algorithms targeting a broad range of application areas need to be accelerated using Jetson. However, given the mission-critical nature of these applications, a thorough study of security of Jetson is required.

While Jetson is an impressive platform for embedded machine learning, it is by no means the paragon platform. Both established and start-up companies have been recently launching novel embedded accelerators for machine-learning. Further, researchers have been exploring unconventional memories such as non-volatile memories and processing-in-memory approaches [125–127], which avoid data-transfer overheads altogether. Evidently, the platform of choice will be decided based on a variety of metrics such as cost, latency, throughput, energy-efficiency, support of software libraries, and more. Most works reviewed here have evaluated only few of these metrics. Evidently, the race to optimization has not finished at all, but seems to have begun just recently. No doubt, processor-design industry has an exciting future ahead!

REFERENCES

[1] https://rebootingcomputing.ieee.org/lpirc, 2018.

[2] S. Alyamkin, M. Ardi, A. Brighton, A. C. Berg, Y. Chen, H.-P. Cheng, B. Chen, Z. Fan, C. Feng, B. Fu *et al.*, "2018 low-power image recognition challenge," *arXiv preprint arXiv:1810.01732*, 2018.

[3] S. Mittal and J. Vetter, "A Survey of CPU-GPU Heterogeneous Computing Techniques," *ACM Computing Surveys*, vol. 47, no. 4, pp. 69:1–69:35, 2015.

[4] C.-K. Lai, C.-W. Yeh, C.-H. Tu, and S.-H. Hung, "Fast profiling framework and race detection for heterogeneous system," *Journal of Systems Architecture*, vol. 81, pp. 83–91, 2017.

[5] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot multibox detector," in *European conference on computer vision*, 2016, pp. 21–37.

[6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *IEEE conference on computer vision and pattern recognition*, 2016, pp. 779–788.

[7] D. Kang, D. Kang, J. Kang, S. Yoo, and S. Ha, "Joint optimization of speed, accuracy, and energy for embedded image recognition systems," in *2018 Design, Automation Test in Europe Conference Exhibition (DATE)*, March 2018, pp. 715–720.

[8] L. Fridman, D. E. Brown, M. Glazer, W. Angell, S. Dodd, B. Jenik, J. Terwilliger, J. Kindelsberger, L. Ding, S. Seaman *et al.*, "Mit autonomous vehicle technology study: Large-scale deep learning based analysis of driver behavior and interaction with automation," *arXiv preprint arXiv:1711.06976*, 2017.

[9] https://elinux.org/Jetson.

[10] https://devblogs.nvidia.com/jetson-tx2-delivers-twice-intelligence-edge/.

[11] S. Leroux, S. Bohez, E. De Coninck, T. Verbelen, B. Vankeirsbilck, P. Simoens, and B. Dhoedt, "The cascading neural network: building the internet of smart things," *Knowledge and Information Systems*, vol. 52, no. 3, pp. 791–814, 2017.

[12] G. Hegde, Siddhartha, and N. Kapre, "Caffepresso: Accelerating convolutional networks on embedded socs," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 17, no. 1, p. 15, 2018.

[13] X. Zhang, Y. Wang, and W. Shi, "pCAMP: Performance Comparison of Machine Learning Packages on the Edges," *HotEdge*, 2018.

[14] S. Mittal, "A Survey of FPGA-based Accelerators for Convolutional Neural Networks," *Neural computing and applications*, 2018.

[15] "Jetson tk1: Mobile embedded supercomputer takes cuda everywhere," https://devblogs.nvidia.com/jetson-tk1-mobile-embedded-supercomputer-cuda-everywhere/, 2018.

[16] https://devtalk.nvidia.com/default/topic/1024825/cuda-programming-and-performance/jetson-tx2-performance/.

[17] http://web.eece.maine.edu/~vweaver/group/green_machines.html.
[18] https://www.raspberrypi.org/forums/viewtopic.php?t=161698.
[19] "Nvidia jetson tx1 supercomputer-on-module drives next wave of autonomous machines," https://devblogs.nvidia.com/nvidia-jetson-tx1-supercomputer-on-module-drives-next-wave-of-autonomous-machines/.
[20] "Nvidia jetson tk1 review," https://www.itpro.co.uk/desktop-hardware/22731/nvidia-jetson-tk1-review.
[21] "Nvidia jetson tx1 system-on-module," http://images.nvidia.com/content/tegra/embedded-systems/pdf/JTX1-Module-Product-sheet.pdf.
[22] "New product: Raspberry pi 3 model b," https://www.pololu.com/blog/598/new-product-raspberry-pi-3-model-b.
[23] "Up board 4gb ram + 64 gb emmc," https://up-shop.org/up-boards/44-up-board-4gb-ram-64-gb-emmc.html.
[24] N. Tijtgat, W. Van Ranst, B. Volckaert, T. Goedemé, and F. De Turck, "Embedded real-time object detection for a uav warning system," in *ICCV2017, the International Conference on Computer Vision*, 2017, pp. 2110–2118.
[25] S. Mittal, "Power management techniques for data centers: A survey," Oak Ridge National Laboratory, USA, Tech. Rep. ORNL/TM-2014/381, 2014.
[26] S. Mittal and J. S. Vetter, "A Survey of Methods for Analyzing and Improving GPU Energy Efficiency," *ACM Computing Surveys*, vol. 47, no. 2, pp. 19:1–19:23, 2015.
[27] C. E. Kim, M. M. D. Oghaz, J. Fajtl, V. Argyriou, and P. Remagnino, "A comparison of embedded deep learning methods for person detection," *arXiv preprint arXiv:1812.03451*, 2018.
[28] "Nvidia jetson agx xavier developer kit," https://developer.nvidia.com/embedded/buy/jetson-xavier-devkit, 2018.
[29] P. S. Paolucci, R. Ammendola, A. Biagioni, O. Frezza, F. L. Cicero, A. Lonardo, M. Martinelli, E. Pastorelli, F. Simula, and P. Vicini, "Power, energy and speed of embedded and server multi-cores applied to distributed simulation of spiking neural networks: Arm in nvidia tegra vs intel xeon quad-cores," *arXiv preprint arXiv:1505.03015*, 2015.
[30] B. Reddy, Y.-H. Kim, S. Yun, C. Seo, and J. Jang, "Real-time driver drowsiness detection for embedded system using model compression of deep neural networks," *Comput. Vis. Pattern Recognit. Work*, 2017.
[31] D. Tomè, F. Monti, L. Baroffio, L. Bondi, M. Tagliasacchi, and S. Tubaro, "Deep convolutional neural networks for pedestrian detection," *Signal processing: image communication*, vol. 47, pp. 482–489, 2016.
[32] S. Rallapalli, H. Qiu, A. Bency, S. Karthikeyan, R. Govindan, B. Manjunath, and R. Urgaonkar, "Are very deep neural networks feasible on mobile devices," *IEEE Trans. Circ. Syst. Video Technol*, 2016.
[33] I. Ardiyanto, H. A. Nugroho, and R. L. B. Buana, "Deep learning-based diabetic retinopathy assessment on embedded system," in *Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, 2017, pp. 1760–1763.
[34] Q. Chen, R. Luley, Q. Wu, M. Bishop, R. W. Linderman, and Q. Qiu, "AnRAD: a neuromorphic anomaly detection framework for massive concurrent data streams," *IEEE transactions on neural networks and learning systems*, vol. 29, no. 5, pp. 1622–1636, 2018.
[35] L. Ran, Y. Zhang, T. Yang, and T. Chen, "Autonomous near ground quadrone navigation with uncalibrated spherical images using convolutional neural networks," in *International Conference on Advances in Mobile Computing and Multi Media*, 2016, pp. 342–347.
[36] N. Deepika and V. S. Variyar, "Obstacle classification and detection for vision based navigation for autonomous driving," in *Advances in Computing, Communications and Informatics (ICACCI), 2017 International Conference on*, 2017, pp. 2092–2097.
[37] A. Page, C. Shea, and T. Mohsenin, "Wearable seizure detection using convolutional neural networks with transfer learning," in *International Symposium on Circuits and Systems (ISCAS)*, 2016, pp. 1086–1089.
[38] S. M. A. H. Jafri, A. Hemani, and L. Intesa, "SPEED: Open-source framework to accelerate speech recognition on embedded GPUs," in *Euromicro Conference on Digital System Design (DSD)*, 2017, pp. 94–101.
[39] X. Xie, D. Du, Q. Li, Y. Liang, W. T. Tang, Z. L. Ong, M. Lu, H. P. Huynh, and R. S. M. Goh, "Exploiting sparsity to accelerate fully connected layers of cnn-based applications on mobile socs," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 17, no. 2, p. 37, 2018.
[40] L. Cavigelli, M. Magno, and L. Benini, "Accelerating real-time embedded scene labeling with convolutional networks," in *Design Automation Conference*. ACM, 2015, p. 108.
[41] N. Smolyanskiy, A. Kamenev, J. Smith, and S. Birchfield, "Toward low-flying autonomous mav trail navigation using deep neural networks for environmental awareness," in *Intelligent Robots and Systems (IROS), 2017 IEEE/RSJ International Conference on*. IEEE, 2017, pp. 4241–4247.
[42] A. Stanoev, N. Audinet, S. Tancock, and N. Dahnoun, "Real-time stereo vision for collision detection on autonomous UAVs," in *Imaging Systems and Techniques (IST), 2017 IEEE International Conference on*, 2017, pp. 1–6.
[43] Susanto, E. Rudiawan, R. Analia, D. S. P, and H. Soebakti, "The deep learning development for real-time ball and goal detection of Barelang-FC," in *Engineering Technology and Applications (IES-ETA), 2017 International Electronics Symposium on*, 2017, pp. 146–151.
[44] S. Lian, Y. Han, Y. Wang, Y. Bao, H. Xiao, X. Li, and N. Sun, "Dadu: Accelerating Inverse Kinematics for High-DOF Robots," in *Design Automation Conference (DAC)*, 2017, pp. 1–6.
[45] R. Yazdani, A. Segura, J.-M. Arnau, and A. Gonzalez, "Low-power automatic speech recognition through a mobile gpu and a viterbi accelerator," *IEEE micro*, vol. 37, no. 1, pp. 22–29, 2017.
[46] H. Lee, J. Kim, D. Yang, and J.-H. Kim, "Embedded real-time fall detection using deep learning for elderly care," *arXiv preprint arXiv:1711.11200*, 2017.
[47] X. Zeng, K. Cao, and M. Zhang, "MobileDeepPill: A small-footprint mobile deep learning system for recognizing unconstrained pill images," in *International Conference on Mobile Systems, Applications, and Services (MobiSys)*, 2017, pp. 56–67.
[48] L.-Y. Duan, W. Sun, X. Zhang, S. Wang, J. Chen, J. Yin, S. See, T. Huang, A. C. Kot, and W. Gao, "Fast MPEG-CDVS Encoder With GPU-CPU Hybrid Computing," *IEEE Transactions on Image Processing*, vol. 27, no. 5, pp. 2201–2216, 2018.

[49] S. Gu, X. Chen, W. Zeng, and X. Wang, "A deep learning tennis ball collection robot and the implementation on nvidia jetson tx1 board," in *International Conference on Advanced Intelligent Mechatronics (AIM)*, 2018, pp. 170–175.

[50] A. Shakeel, "Service robot for the visually impaired: Providing navigational assistance using deep learning," *Masters' thesis*, 2017.

[51] N. Otterness, M. Yang, S. Rust, E. Park, J. H. Anderson, F. D. Smith, A. Berg, and S. Wang, "An evaluation of the NVIDIA TX1 for supporting real-time computer-vision workloads," in *Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2017, pp. 353–364.

[52] C. Wang, Y. Wang, Y. Han, L. Song, Z. Quan, J. Li, and X. Li, "Cnn-based object detection solutions for embedded heterogeneous multicore socs," in *Design Automation Conference (ASP-DAC), 2017 22nd Asia and South Pacific*, 2017, pp. 105–110.

[53] Y.-C. Tsai, K.-H. Chen, Y. Chen, and J.-H. Cheng, "Accurate and fast obstacle detection method for automotive applications based on stereo vision," in *International Symposium on VLSI Design, Automation and Test (VLSI-DAT)*. IEEE, 2018, pp. 1–4.

[54] N. Attaran, A. Puranik, J. Brooks, and T. Mohsenin, "Embedded low-power processor for personalized stress detection," *IEEE Transactions on Circuits and Systems II: Express Briefs*, 2018.

[55] T. Abtahi, C. Shea, A. Kulkarni, and T. Mohsenin, "Accelerating Convolutional Neural Network With FFT on Embedded Hardware," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 26, no. 9, pp. 1737–1749, 2018.

[56] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning convolutional neural networks for resource efficient inference," *arXiv preprint arXiv:1611.06440*, 2016.

[57] Y. Wang, Z. Lai, G. Huang, B. H. Wang, L. van der Maaten, M. Campbell, and K. Q. Weinberger, "Anytime stereo image depth estimation on mobile devices," *arXiv preprint arXiv:1810.11408*, 2018.

[58] I. Sa, Z. Chen, M. Popović, R. Khanna, F. Liebisch, J. Nieto, and R. Siegwart, "WeedNet: Dense Semantic Weed Classification Using Multispectral Images and MAV for Smart Farming," *IEEE Robotics and Automation Letters*, vol. 3, no. 1, pp. 588–595, 2018.

[59] R. Hadidi, J. Cao, M. Woodward, M. Ryoo, and H. Kim, "Musical Chair: Efficient Real-Time Recognition Using Collaborative IoT Devices," *arXiv preprint arXiv:1802.02138*, 2018.

[60] M. Gamal, M. Siam, and M. Abdel-Razek, "Shuffleseg: Real-time semantic segmentation network," *arXiv preprint arXiv:1803.03816*, 2018.

[61] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke, "Sim-to-real: Learning agile locomotion for quadruped robots," *arXiv preprint arXiv:1804.10332*, 2018.

[62] R. Madaan, D. Maturana, and S. Scherer, "Wire detection using synthetic data and dilated convolutional networks for unmanned aerial vehicles," in *Intelligent Robots and Systems (IROS), 2017 IEEE/RSJ International Conference on*. IEEE, 2017, pp. 3487–3494.

[63] S. Sadiq, M. Zmieva, M.-L. Shyu, and S.-C. Chen, "Reduced residual nets (red-nets): Low powered adversarial outlier detectors," in *2018 IEEE International Conference on Information Reuse and Integration (IRI)*. IEEE, 2018, pp. 436–443.

[64] A. Hartwell, V. Kadirkamanathan, and S. R. Anderson, "Compact deep neural networks for computationally efficient gesture classification from electromyography signals," *arXiv preprint arXiv:1806.08641*, 2018.

[65] B. Taylor, V. S. Marco, W. Wolff, Y. Elkhatib, and Z. Wang, "Adaptive deep learning model selection on embedded systems," in *International Conference on Languages, Compilers, and Tools for Embedded Systems*, 2018, pp. 31–43.

[66] I. Baek, A. Davies, G. Yan *et al.*, "Real-time detection, tracking, and classification of moving and stationary objects using multiple fisheye images," *arXiv preprint arXiv:1803.06077*, 2018.

[67] N. Bhandary, C. MacKay, A. Richards, J. Tong, and D. C. Anastasiu, "Robust classification of city roadway objects for traffic related applications," in *IEEE SmartWorld/SCALCOM/UIC/ATC/CBDCom/IOP/SCI*, 2017, pp. 1–6.

[68] J. Baek, I. V. Chelu, L. Iordache, V. Paunescu, H. Ryu, A. Ghiuta, A. Petreanu, Y. Soh, A. Leica, and B. Jeon, "Scene understanding networks for autonomous driving based on around view monitoring system," *arXiv preprint arXiv:1805.07029*, 2018.

[69] T. Manderson, J. C. G. Higuera, R. Cheng, and G. Dudek, "Vision-based autonomous underwater swimming in dense coral for combined collision avoidance and target selection," *International Conference on Intelligent Robots and Systems (IROS)*, 2018.

[70] M. G. Bechtel, E. McEllhiney, and H. Yun, "DeepPicar: A Low-cost Deep Neural Network-based Autonomous Car," *arXiv preprint arXiv:1712.08644*, 2017.

[71] N. J. Sanket, C. Singh, K. Ganguly, C. Fermuller, and Y. Aloimonos, "GapFlyt: Active Vision Based Minimalist Structure-less Gap Detection For Quadrotor Flight," *IEEE Robotics and Automation Letters*, 2018.

[72] X. Xu, X. Zhang, B. Yu, X. S. Hu, C. Rowen, J. Hu, and Y. Shi, "DAC-SDC Low Power Object Detection Challenge for UAV Applications," *arXiv preprint arXiv:1809.00110*, 2018.

[73] P. Ghazi, A. P. Happonen, J. Boutellier, and H. Huttunen, "Embedded implementation of a deep learning smile detector," *arXiv preprint arXiv:1807.10570*, 2018.

[74] D. Lee, J.-S. Lee, S. Lee, and S.-C. Kee, "The real-time implementation for the parking line departure warning system," in *2018 3rd IEEE International Conference on Intelligent Transportation Engineering (ICITE)*. IEEE, 2018, pp. 236–240.

[75] J. Li, F. Zhou, and T. Ye, "Real-world railway traffic detection based on faster better network," *IEEE Access*, 2018.

[76] S. Lin, J. Wang, W. Yang, and G. Xia, "Toward autonomous rotation-aware unmanned aerial grasping," *arXiv preprint arXiv:1811.03921*, 2018.

[77] Y. Sun, X. Liu, M. Yuan, L. Ren, J. Wang, and Z. Chen, "Automatic in-trap pest detection using deep learning for pheromone-based dendroctonus valens monitoring," *Biosystems Engineering*, vol. 176, pp. 140–150, 2018.

[78] X. Qi, C. Liu, and S. Schuckers, "IoT Edge Device Based Key Frame Extraction for Face in Video Recognition," in *2018 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*. IEEE, 2018, pp. 641–644.

[79] U. K. Sreekumar, R. Devaraj, Q. Li, and K. Liu, "TPCAM: Real-time traffic pattern collection and analysis model based on deep learning," in *IEEE SmartWorld/SCALCOM/UIC/ATC/CBDCom/IOP/SCI*, 2017, pp. 1–4.

[80] S. Cao, Y. Liu, P. Lasang, and S. Shen, "Detecting the objects on the road using modular lightweight network," *arXiv preprint arXiv:1811.06641*, 2018.

[81] S. Jung, S. Hwang, H. Shin, and D. H. Shim, "Perception, guidance, and navigation for indoor autonomous drone racing using deep learning," *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 2539–2544, 2018.

[82] T. Yang, Q. Ren, F. Zhang, B. Xie, H. Ren, J. Li, and Y. Zhang, "Hybrid Camera Array-Based UAV Auto-Landing on Moving UGV in GPS-Denied Environment," *Remote Sensing*, vol. 10, no. 11, p. 1829, 2018.

[83] S. M. Azimi, "ShuffleDet: Real-Time Vehicle Detection Network in On-board Embedded UAV Imagery," *arXiv preprint arXiv:1811.06318*, 2018.

[84] P. Regier, A. Milioto, P. Karkowski, C. Stachniss, and M. Bennewitz, "Classifying obstacles and exploiting knowledge about classes for efficient humanoid navigation," *International Conference on Humanoid Robots*, 2018.

[85] J. M. Pierre, "Spatio-temporal deep learning for robotic visuomotor control," in *2018 4th International Conference on Control, Automation and Robotics (ICCAR)*. IEEE, 2018, pp. 94–103.

[86] G. Borghi, R. Gasparini, R. Vezzani, and R. Cucchiara, "Embedded recurrent network for head pose estimation in car," in *Proceedings of the 28th IEEE Intelligent Vehicles Symposium*, 2017.

[87] A. Biddulph, T. Houlistion, A. Mendes, and S. K. Chalup, "Comparing computing platforms for deep learning on a humanoid robot," *arXiv preprint arXiv:1809.03668*, 2018.

[88] B. D. Rouhani, A. Mirhoseini, and F. Koushanfar, "DeLight: Adding energy dimension to deep neural networks," in *Proceedings of the 2016 International Symposium on Low Power Electronics and Design*, 2016, pp. 112–117.

[89] A. Jafari, A. Ganesan, C. S. K. Thalisetty, V. Sivasubramanian, T. Oates, and T. Mohsenin, "Sensornet: A scalable and low-power deep convolutional neural network for multimodal data classification," *IEEE Transactions on Circuits and Systems I: Regular Papers*, no. 99, pp. 1–14, 2018.

[90] H. Bura, N. Lin, N. Kumar, S. Malekar, S. Nagaraj, and K. Liu, "An edge based smart parking solution using camera networks and deep learning," in *2018 IEEE International Conference on Cognitive Computing (ICCC)*. IEEE, 2018, pp. 17–24.

[91] S. Luo, H. Lu, J. Xiao, Q. Yu, and Z. Zheng, "Robot detection and localization based on deep learning," in *Chinese Automation Congress (CAC), 2017*, 2017, pp. 7091–7095.

[92] D. Kang, E. Kim, I. Bae, B. Egger, and S. Ha, "C-GOOD: C-code generation framework for optimized on-device deep learning," in *International Conference on Computer-Aided Design*, 2018, p. 105.

[93] A. Montanari, F. Kringberg, A. Valentini, C. Mascolo, and A. Prorok, "Surveying areas in developing regions through context aware drone mobility," in *ACM Workshop on Micro Aerial Vehicle Networks, Systems, and Applications*, 2018, pp. 27–32.

[94] A. Carrio, S. Vemprala, A. Ripoll, S. Saripalli, and P. Campoy, "Drone detection using depth maps," *arXiv preprint arXiv:1808.00259*, 2018.

[95] S. A. Miraftabzadeh, P. Rad, K.-K. R. Choo, and M. Jamshidi, "A privacy-aware architecture at the edge for autonomous real-time identity re-identification in crowds," *IEEE Internet of Things Journal*, 2017.

[96] J. Wang, Z. Feng, Z. Chen, S. George, M. Bala, P. Pillai, S.-W. Yang, and M. Satyanarayanan, "Bandwidth-efficient live video analytics for drones via edge computing," *Symposium on Edge Computing (SEC)*, 2018.

[97] P.-E. Sarlin, F. Debraine, M. Dymczyk, R. Siegwart, and C. Cadena, "Leveraging deep visual descriptors for hierarchical efficient localization," *arXiv preprint arXiv:1809.01019*, 2018.

[98] M. Goyal, N. Reeves, S. Rajbhandari, and M. H. Yap, "Robust methods for real-time diabetic foot ulcer detection and localization on mobile devices," *IEEE journal of biomedical and health informatics*, 2018.

[99] J. Kaster, J. Patrick, and H. S. Clouse, "Convolutional neural networks on small unmanned aerial systems," in *Aerospace and Electronics Conference (NAECON), 2017 IEEE National*. IEEE, 2017, pp. 149–154.

[100] E. Ilg, N. Mayer, T. Saikia, M. Keuper, A. Dosovitskiy, and T. Brox, "Flownet 2.0: Evolution of optical flow estimation with deep networks," in *IEEE conference on computer vision and pattern recognition (CVPR)*, vol. 2, 2017, p. 6.

[101] S. Mittal, "A survey of techniques for architecting and managing asymmetric multicore processors," *ACM Computing Surveys*, vol. 48, no. 3, pp. 45:1–45:38, 2016.

[102] S. Mittal, "A survey of techniques for improving energy efficiency in embedded computing systems," *International Journal of Computer Aided Engineering and Technology (IJCAET)*, vol. 6, no. 4, pp. 440–459, 2014.

[103] S. Mittal, "A survey of techniques for approximate computing," *ACM Computing Surveys*, vol. 48, no. 4, pp. 62:1–62:33, 2016.

[104] L. Cavigelli, P. Degen, and L. Benini, "CBinfer: Change-based inference for convolutional neural networks on video data," in *International Conference on Distributed Smart Cameras*, 2017, pp. 1–8.

[105] Z. Wang, W. Ren, and Q. Qiu, "LaneNet: Real-Time Lane Detection Networks for Autonomous Driving," *arXiv preprint arXiv:1807.01726*, 2018.

[106] L. Cai, A.-M. Barneche, A. Herbout, C. S. Foo, J. Lin, V. R. Chandrasekhar, and M. M. Sabry, "TEA-DNN: the Quest for Time-Energy-Accuracy Co-optimized Deep Neural Networks," *arXiv preprint arXiv:1811.12065*, 2018.

[107] T. T. Duong, J. H. Seo, T. D. Tran, B. J. Young, and J. W. Jeon, "Evaluation of embedded systems for automotive image processing," in *2018 19th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*. IEEE, 2018, pp. 123–128.

[108] S. Mittal, "A Survey of Techniques for Architecting and Managing GPU Register File," *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, 2016.

[109] Q. Li, Q. Xiao, and Y. Liang, "Enabling high performance deep learning networks on embedded systems," in *Industrial Electronics Society, IECON 2017-43rd Annual Conference of the IEEE*. IEEE, 2017, pp. 8405–8410.

[110] S. Mittal and J. Vetter, "A Survey Of Architectural Approaches for Data Compression in Cache and Main Memory Systems," *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, 2015.

[111] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Advances in neural information processing systems*, 2014, pp. 3320–3328.

[112] S. Liu, S. De Mello, J. Gu, G. Zhong, M.-H. Yang, and J. Kautz, "Learning affinity via spatial propagation networks," in

Advances in Neural Information Processing Systems, 2017, pp. 1520–1530.
[113] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *NIPS*, 2014.
[114] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint face detection and alignment using multitask cascaded convolutional networks," *IEEE Signal Processing Letters*, vol. 23, no. 10, pp. 1499–1503, 2016.
[115] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Advances in neural information processing systems*, 2015, pp. 91–99.
[116] J. Pedoeem and R. Huang, "Yolo-lite: A real-time object detection algorithm optimized for non-gpu computers," *arXiv preprint arXiv:1811.05588*, 2018.
[117] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
[118] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Conference on Computer Vision and Pattern Recognition*, 2001.
[119] Y. Jang, H. Gunes, and I. Patras, "SmileNet: Registration-Free Smiling Face Detection In The Wild," 2018.
[120] V. Badrinarayanan, A. Kendall, and R. Cipolla, "Segnet: A deep convolutional encoder-decoder architecture for image segmentation," *IEEE Transactions on Pattern Analysis & Machine Intelligence*, no. 12, pp. 2481–2495, 2017.
[121] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.
[122] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," in *Advances in Neural Information Processing Systems*, 2015, pp. 2692–2700.
[123] S. Mittal, "A survey of architectural techniques for improving cache power efficiency," *Sustainable Computing: Informatics and Systems*, vol. 4, no. 1, pp. 33–43, 2014.
[124] S. Mittal and S. Nag, "A survey of encoding techniques for reducing data-movement energy," *Journal of Systems Architecture*, 2018.
[125] S. Umesh and S. Mittal, "A survey of spintronic architectures for processing-in-memory and neural networks," *Journal of Systems Architecture*, 2018.
[126] S. Mittal, "A Survey of ReRAM-based Architectures for Processing-in-memory and Neural Networks," *Machine learning and knowledge extraction*, vol. 1, p. 5, 2018.
[127] M. Scrbak, M. Islam, K. M. Kavi, M. Ignatowski, and N. Jayasena, "Exploring the processing-in-memory design space," *Journal of Systems Architecture*, vol. 75, pp. 59–67, 2017.